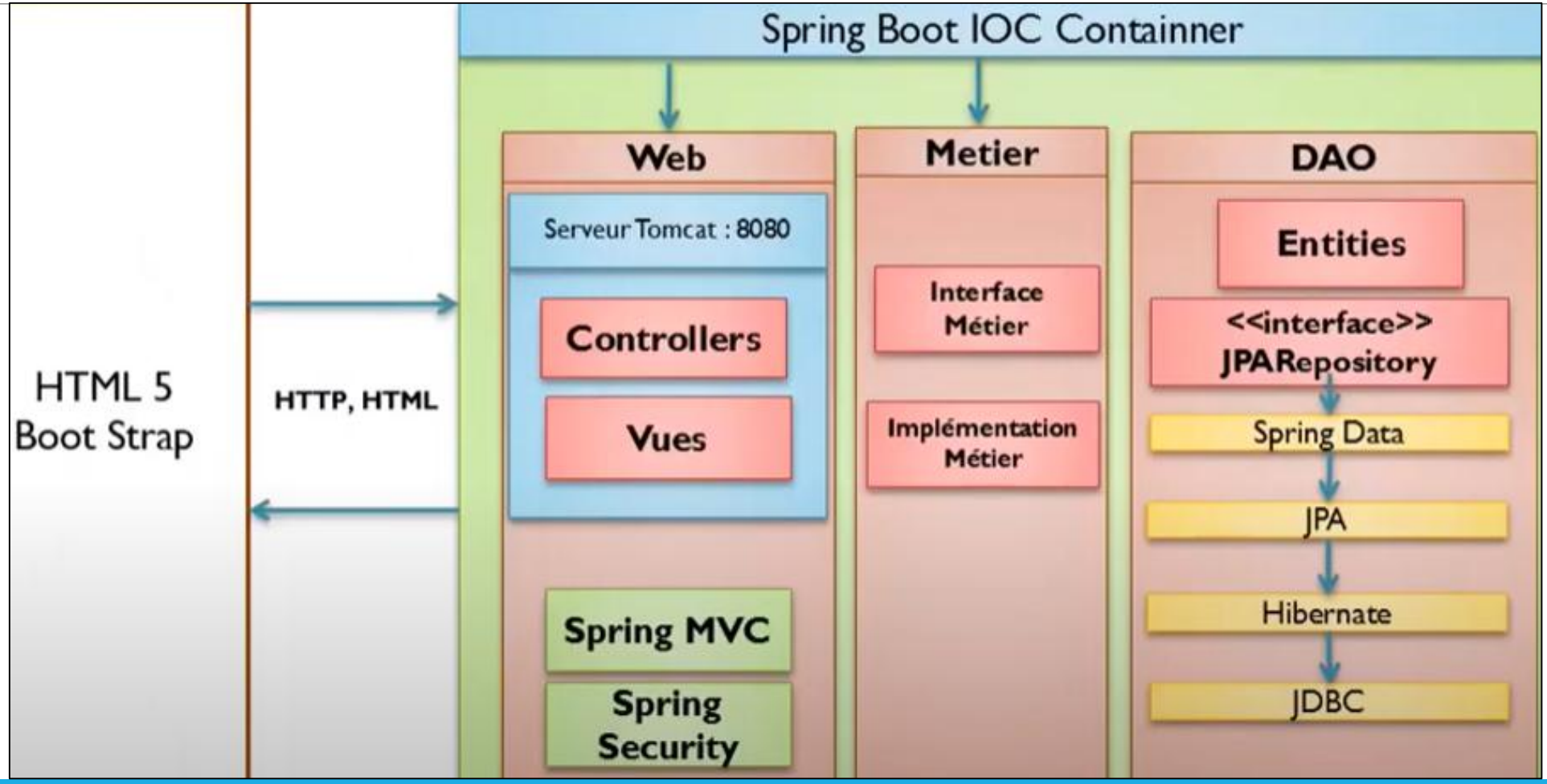
DEVELOPPEMENT D'APPLICATION WEB BASE SUR SPRING MVC

PROJET : GESTION DE PATRIMOINE TOURISITQUE

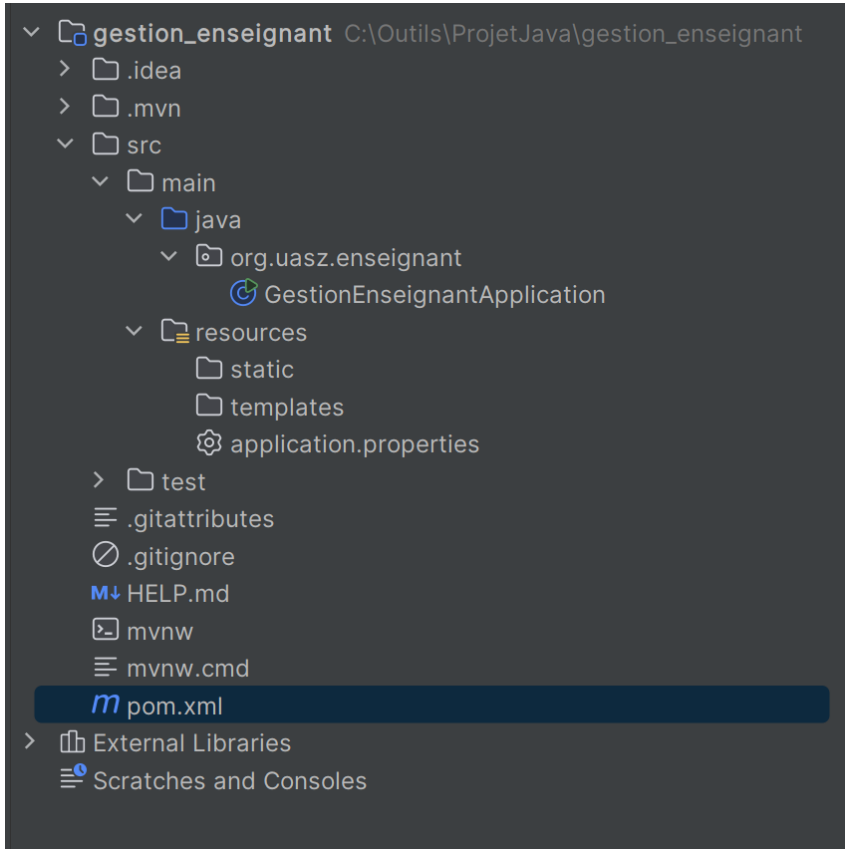
Architecture technique

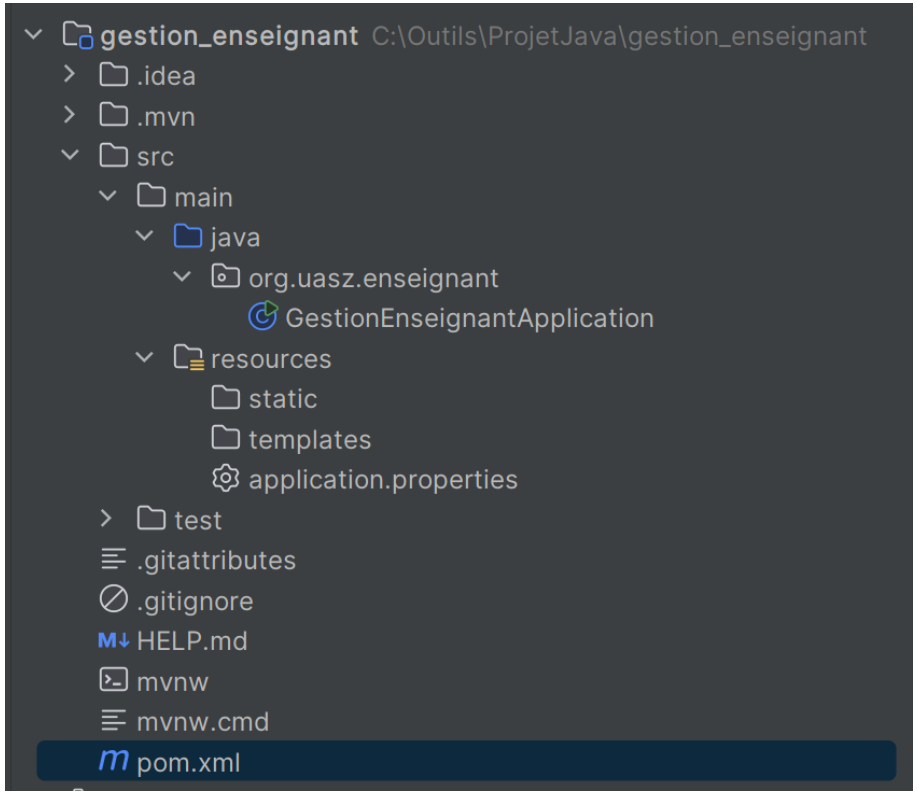


Installation des dependances

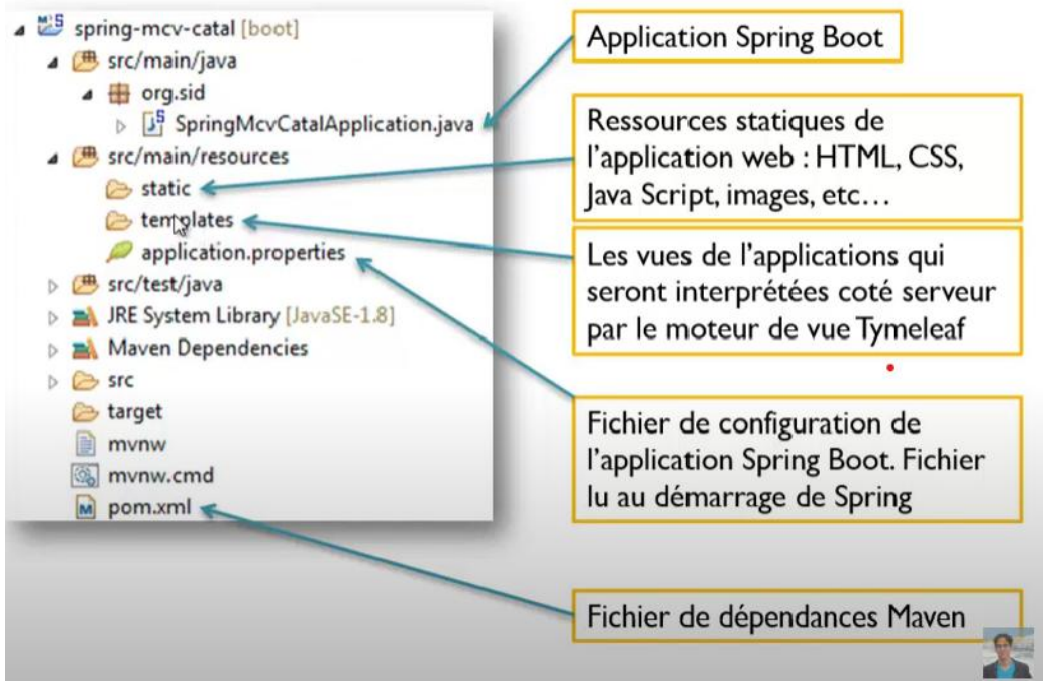
- ✓ Web
- ✓ Jpa
- ✓ Mysql Driver
- ✓ Thymeleaf

Structure du projet





Structure du projet



Entites

UFR (code, libelle)

Departement (code, libelle, #ufr)

Grade(libelle)

Enseignant (matricule, prenom, nom, dateNaiss, lieuNais, civilite,isActive, statut)

```
@Entity  no usages
public class Enseignant implements Serializable {
    @Id  2 usages
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String matricule;  3 usages
    private String prenom;  3 usages
    private String nom;  3 usages
    private Date dateNaissance;  3 usages
    private String lieuNaissance;  3 usages
    private Civilite civilite;  3 usages
    private Statut statut;  3 usages
    private boolean isActive;  3 usages
```

Constructeurs, Getter et Setter

Génération de construction sans paramètre et construction avec paramètre

Generation des getter et setter

Connexion à la base de données

Configuration des paramètres de connexion à la base de données : **application.properties**

```
spring.application.name=gestion_enseignant
spring.datasource.url=jdbc:mysql://localhost:3306/gestion_enseignant
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect = org.hibernate.dialect.MySQLDialect
```

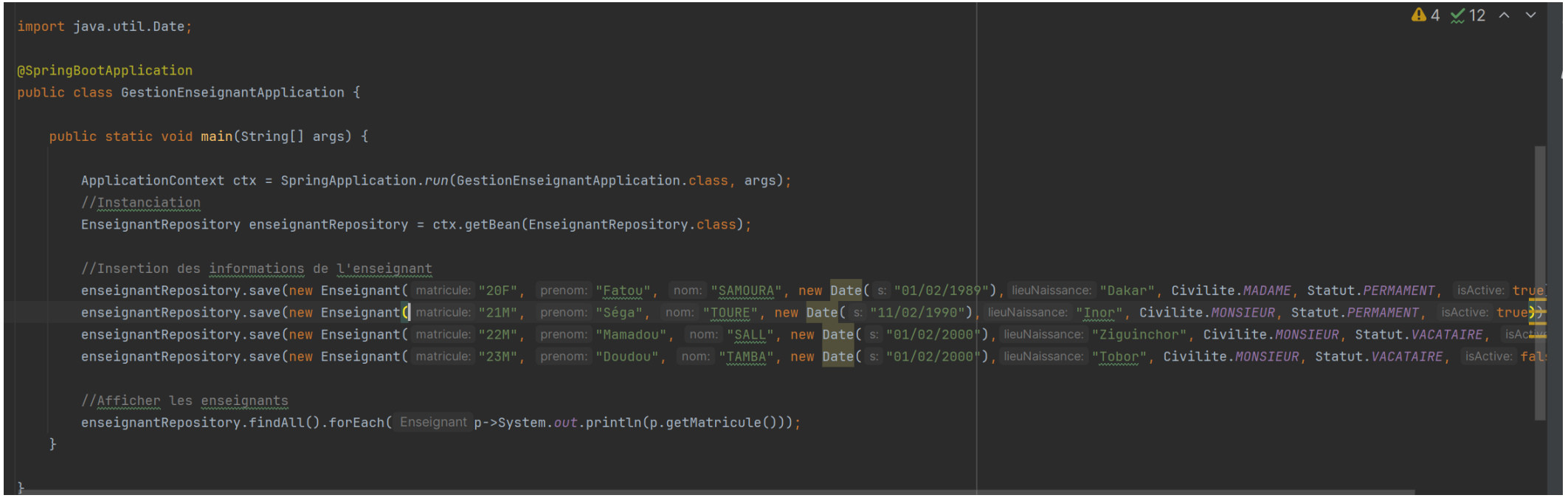
Utilisation de JPA

```java
package org.uasz.enseignant.dao;

import org.springframework.data.jpa.repository.JpaRepository;
import org.uasz.enseignant.entities.Enseignant;

public interface EnseignantRepository extends JpaRepository<Enseignant, Long> {  no usages
}
```

Tester à la methode main()



```java
import java.util.Date;

@SpringBootApplication
public class GestionEnseignantApplication {

    public static void main(String[] args) {

        ApplicationContext ctx = SpringApplication.run(GestionEnseignantApplication.class, args);
        //Instanciation
        EnseignantRepository enseignantRepository = ctx.getBean(EnseignantRepository.class);

        //Insertion des informations de l'enseignant
        enseignantRepository.save(new Enseignant( matricule: "20F",  prenom: "Fatou",  nom: "SAMOURA", new Date( s: "01/02/1989"), lieuNaissance: "Dakar", Civilite.MADAME, Statut.PERMAMENT,  isActive: true
        enseignantRepository.save(new Enseignant( matricule: "21M",  prenom: "Séga",  nom: "TOURE", new Date( s: "11/02/1990"), lieuNaissance: "Inor", Civilite.MONSIEUR, Statut.PERMAMENT,  isActive: true));
        enseignantRepository.save(new Enseignant( matricule: "22M",  prenom: "Mamadou",  nom: "SALL", new Date( s: "01/02/2000"), lieuNaissance: "Ziguinchor", Civilite.MONSIEUR, Statut.VACATAIRE,  isAc
        enseignantRepository.save(new Enseignant( matricule: "23M",  prenom: "Doudou",  nom: "TAMBA", new Date( s: "01/02/2000"), lieuNaissance: "Tobor", Civilite.MONSIEUR, Statut.VACATAIRE,  isActive: fal

        //Afficher les enseignants
        enseignantRepository.findAll().forEach( Enseignant p->System.out.println(p.getMatricule()));
    }

}
```

Création de controller

1 – Methode Inde

```java
@Controller  no usages
public class EnseignantController {
    @Autowired  1 usage
    private EnseignantRepository enseignantRepository;

    @RequestMapping(value = "/index")  no usages
    public String index(Model model){
        List<Enseignant> enseignants = enseignantRepository.findAll();
        model.addAttribute( attributeName: "listEnseigants",enseignants);
        return "enseignants";
    }

}
```

Creation de vue

Chemin => resources

- => templates
  - => enseignants.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymleaf.org" lang="en">
<head>
    <meta charset="UTF-8"/>
    <title>Enseignants</title>
</head>
<body>
    <h3>Test</h3>
    <table border="1">
        <thead>
            <tr>
                <th>Id</th><th>Civilité</th><th>Matricule</th><th>Prénom</th><th>Nom</th><th>Date de naissance</th><th>Lieu de naissance</th><th>Statut</th><th>Activer</th>
            </tr>
        </thead>
        <tbody>
            <tr th:each="p:${listEnseigants}">
                <td th:text="${p.id}"></td>
                <td th:text="${p.civilite}"></td>
                <td th:text="${p.matricule}"></td>
                <td th:text="${p.prenom}"></td>
                <td th:text="${p.nom}"></td>
                <td th:text="${p.dateNaissance}"></td>
                <td th:text="${p.lieuNaissance}"></td>
                <td th:text="${p.statut}"></td>
                <td th:text="${p.isActive}"></td>
            </tr>
        </tbody>
    </table>
</body>
</html>
```

Utilisation de bootstrap 5

Url => https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css

Lien

```
<link rel="stylesheet" type="text/css" href="../static/css/bootstrap.min.css">
```

Pagination

```java
@RequestMapping(value = "/index")  no usages
public String index(Model model,
                    @RequestParam(name = "page", defaultValue = "0") int p,
                    @RequestParam(name = "size", defaultValue = "5") int s){
    //Liste enseignants
   // List<Enseignant> enseignants = enseignantRepository.findAll();
    //model.addAttribute("listEnseigants",enseignants);

    //Page de 4 enseignants
    Pageable limit = PageRequest.of(p,s);
    Page<Enseignant> pageEnseignants =
            enseignantRepository.findAll(limit);
    model.addAttribute( attributeName: "listEnseigants",pageEnseignants.getContent());
    return "enseignants";
}
```

http://localhost:8080/index?page=2&size=8

Pagination (gestion page courante)

```java
@RequestMapping(value = "/index")  no usages
public String index(Model model,
                    @RequestParam(name = "page", defaultValue = "0") int p,
                    @RequestParam(name = "size", defaultValue = "5") int s){
    //Liste enseignants
   // List<Enseignant> enseignants = enseignantRepository.findAll();
    //model.addAttribute("listEnseigants",enseignants);

    //Page de 4 enseignants
    Pageable limit = PageRequest.of(p,s);
    Page<Enseignant> pageEnseignants =
            enseignantRepository.findAll(limit);
    model.addAttribute( attributeName: "listEnseigants",pageEnseignants.getContent());
    int[] pages = new int[pageEnseignants.getTotalPages()];
    model.addAttribute( attributeName: "lstPages", pages);
    model.addAttribute( attributeName: "pageCourante", p);
    model.addAttribute( attributeName: "size", s);

    return "enseignants";
}
```

Pagination (gestion page courante)

```
<div class="container">
    <ul class="nav nav-pills">
        <li th:class="${pageCourante}==${status.index}?'active':''" th:each="pa,status:${lstPages}" >
            <a th:href="@{index(page=${status.index},size=${size})}" th:text="${status.index}" class="nav-link"></a>
        </li>
    </ul>
</div>
```

Recherche par mot clés

Controller

```
@RequestMapping(value = "/index")  no usages
public String index(Model model,
                    @RequestParam(name = "page", defaultValue = "0") int p,
                    @RequestParam(name = "size", defaultValue = "5") int s,
                    @RequestParam(name = "motCle", defaultValue = "") String mc ){
    //Liste enseignants
   // List<Enseignant> enseignants = enseignantRepository.findAll();
    //model.addAttribute("listEnseigants",enseignants);

    //Page de 4 enseignants
    Pageable limit = PageRequest.of(p,s);
    Page<Enseignant> pageEnseignants =
            enseignantRepository.rechercher( mc: "%"+mc+"%",limit);
    model.addAttribute( attributeName: "listEnseigants",pageEnseignants.getContent());
    int[] pages = new int[pageEnseignants.getTotalPages()];
    model.addAttribute( attributeName: "lstPages", pages);
    model.addAttribute( attributeName: "pageCourante", p);
    model.addAttribute( attributeName: "size", s);
    model.addAttribute( attributeName: "motCle", mc);
    return "enseignants";
}
```

Recherche par mot clés

Vue

```
<div class="container">
    <form th:action="@{index}" method="get">
        <label>Mot Clé : </label>
        <input type="text" name="motCle" th:value="${motCle}"/>
        <button class="btn btn-primary">Recherche</button>
    </form>
</div>
```

```
<div class="container">
    <ul class="nav nav-pills">
        <li th:class="${pageCourante}==${status.index}?'active':''" th:each="pa,status:${lstPages}" >
            <a th:href="@{index(page=${status.index},size=${size},motCle=${motCle})}" th:text="${status.index}" class="nav-link"></a>
        </li>
    </ul>
</div>
```

Delete

Controller

```
@RequestMapping(value = "/delete", method = RequestMethod.GET)  no usages
public String delete(Long id, String motCle,int page, int size){
    enseignantRepository.deleteById(id);
    return "redirect:/index?page="+page+"&size="+size+"&motCle="+motCle;
}
```

Delete

Vue

```
<tbody>
<tr th:each="p:${listEnseigants}">
    <td th:text="${p.id}"></td>
    <td th:text="${p.civilite}"></td>
    <td th:text="${p.matricule}"></td>
    <td th:text="${p.prenom}"></td>
    <td th:text="${p.nom}"></td>
    <td th:text="${p.dateNaissance}"></td>
    <td th:text="${p.lieuNaissance}"></td>
    <td th:text="${p.statut}"></td>
    <td th:text="${p.isActive}"></td>
    <td><a th:href="@{delete(id=${p.id},page=${pageCourante},size=${size},motCle=${motCle})}">Delete</a></td>
</tr>
</tbody>
```

Create

Controller

```java
@RequestMapping(value = "/create", method = RequestMethod.GET)  no usages
public String create(Model model){
    model.addAttribute( attributeName: "enseignant", new Enseignant());
    return "enseignant-create";
}

@RequestMapping(value = "/save", method = RequestMethod.POST)  no usages
public String save(Model model, @Valid Enseignant enseignant, BindingResult bindingResult){
    if(bindingResult.hasErrors()){
        return "enseignant-create";
    }
    enseignantRepository.save(enseignant);
    return "Confirmation";
}
```

Create

Vue

```
<form th:action="@{save}" method="post">
    <div class="form-group">
        <label class="control-label">Civilité : </label>
        <select class="form-control" name="civilite" th:value="${enseignant.civilite}">
            <option>Choisir</option>
            <option value="Monsieur">Monsieur</option>
            <option value="Madame">Madame</option>
            <option value="Mademoiselle">Mademoiselle</option>
        </select>
        <span class="red" th:errors="${enseignant.civilite}"></span>
    </div>
    <div class="form-group">
        <label class="control-label">Matricule : </label>
        <input class="form-control" type="text" name="matricule" th:value="${enseignant.matricule}"/>
        <span class="red" th:errors="${enseignant.matricule}" ></span>
    </div>
    <div class="form-group">
        <label class="control-label">Prénom : </label>
        <input class="form-control" type="text" name="prenom" th:value="${enseignant.prenom}"/>
        <span class="red" th:errors="${enseignant.prenom}"></span>
    </div>
    <div class="form-group">
        <label class="control-label">Nom : </label>
        <input class="form-control" type="text" name="nom" th:value="${enseignant.nom}"/>
        <span class="red" th:errors="${enseignant.nom}"></span>
    </div>
```

```
    <div class="form-group">
        <label class="control-label">Date naissance : </label>
        <input class="form-control" type="date" name="dateNaissance" th:value="${enseignant.dateNaissance}"/>
        <span class="red" th:errors="${enseignant.dateNaissance}"></span>
    </div>
    <div class="form-group">
        <label class="control-label">Lieu naissance : </label>
        <input class="form-control" type="text" name="lieuNaissance" th:value="${enseignant.lieuNaissance}"/>
        <span class="red" th:errors="${enseignant.lieuNaissance}"></span>
    </div>
    <div class="form-group">
        <label class="control-label">Statut : </label>
        <input type="radio" name="statut"  th:value="Permanent"/>Permanent
        <input type="radio" name="statut" th:value="Vacataire"/>Vacataire
        <span class="red" th:errors="${enseignant.statut}"></span>
    </div>
    <div class="form-group">
        <label class="control-label">Actif : </label>
        <input type="checkbox" name="isActive" th:value="False" />
        <span class="red" th:errors="${enseignant.isActive}"></span>
    </div>
    <div>
        <button type="submit" class="btn btn-primary">Enregistrer</button>
    </div>
</form>
```

Details

Controller

```
@RequestMapping(value = "enseignant/detail/{id}")  no usages
public String detail(@PathVariable Long id, Model model){
    Optional<Enseignant> enseignant = enseignantRepository.findById(id);
    enseignant.ifPresent( Enseignant dataEnseignant -> model.addAttribute( attributeName: "enseignant", dataEnseignant));
    return "enseignant-detail";
}
```

Details

Vue

```
<body>
    <h3>Afficher les détails de l'enseignants</h3>
    <p><strong>ID :</strong> <span th:text="${enseignant.id}"></span></p>
    <p><strong>Civilité :</strong> <span th:text="${enseignant.civilite}"></span></p>
    <p><strong>Matricule :</strong> <span th:text="${enseignant.matricule}"></span></p>
    <p><strong>Prénom :</strong> <span th:text="${enseignant.prenom}"></span></p>
    <p><strong>Nom :</strong> <span th:text="${enseignant.nom}"></span></p>
    <p><strong>Date de naissance :</strong> <span th:text="${enseignant.dateNaissance}"></span></p>
    <p><strong>Lieu de naissance :</strong> <span th:text="${enseignant.lieuNaissance}"></span></p>
    <p><strong>Statut :</strong> <span th:text="${enseignant.statut}"></span></p>
    <p><strong>Actif :</strong> <span th:text="${enseignant.isActive}"></span></p>
</body>
```

Update

Controller

Update

Vue

Layout.html

```html
<html xmlns:th="http://www.thymleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      lang="en">
<head>
    <meta charset="UTF-8"/>
    <title>Gestion des enseigants</title>
    <link rel="stylesheet" type="text/css"  href="../static/css/bootstrap.min.css"  th:href="@{css/bootstrap.min.css}" />
    <link rel="stylesheet" type="text/css"  href="../static/css/style.css"  th:href="@{css/style.css}" />
</head>
<body>
    <header>
        <div class="navbar navbar-default">
            <div class="container-fluid">
                <ul class="nav navbar-nav">
                    <li><a th:href="@{index}">Chercher</a></li>
                    <li><a th:href="@{form}">Enseignants</a></li>
                </ul>
            </div>
        </div>
    </header>
    <section layout:fragment="content">

    </section>
    <footer class="navbar-fixed-bottom">
        <hr />
        <div class="container">
            <small>Copyright @2025</small>
        </div>
    </footer>
</body>
</html>
```

Layout : Decoration des pages

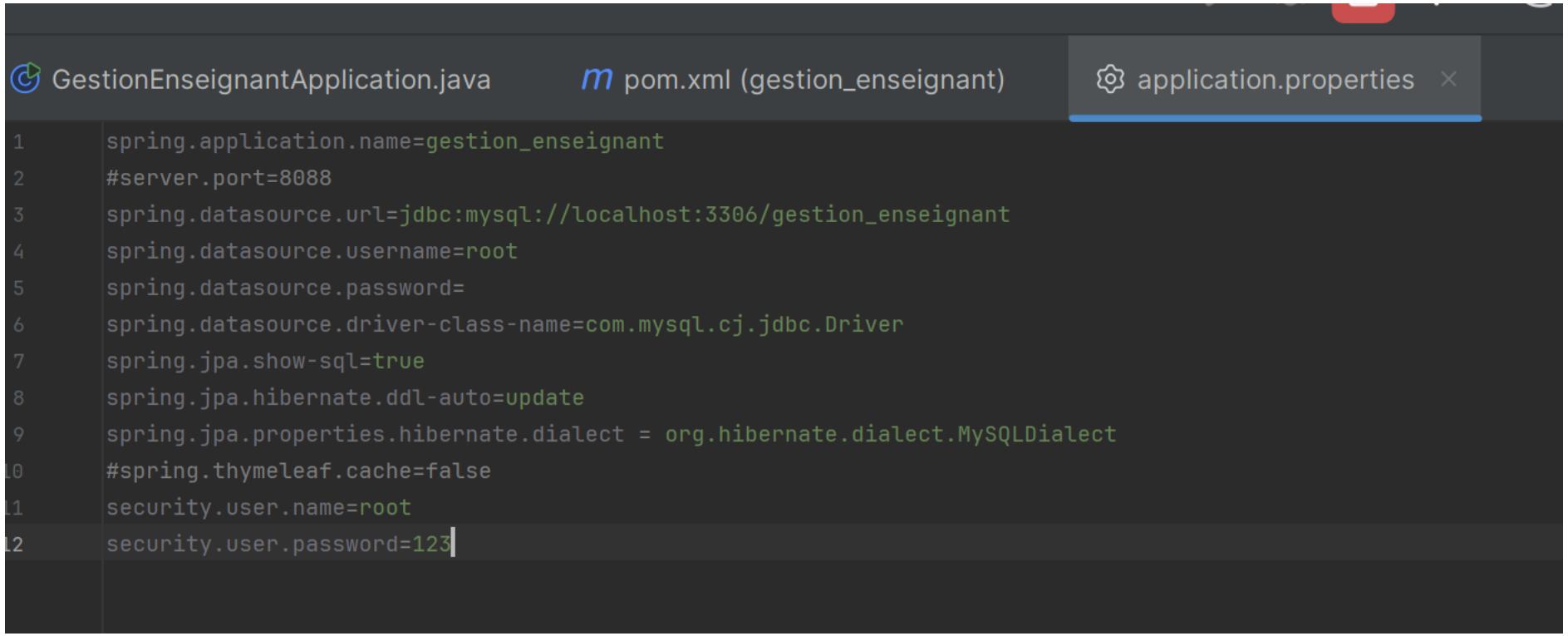
```
<html xmlns:th="http://www.thymleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorator="layout"
      lang="en">
```

```
<div class="container" layout:fragment="content">
    <div>
        <form th:action="@{index}" method="get">
            <label>Mot Clé : </label>
            <input type="text" name="motCle" th:value="${motCle}"/>
            <button class="btn btn-primary">Recherche</button>
        </form>
    </div>
    <div>
        <a th:href="@{/create}">Ajouter enseignant</a>
    </div>

</div>
```

Authenticate avec **Spring Security**

Initilisation des users



Créez une Application Web Sécurisée avec Spring Boot et Thymeleaf

Implémenter les opérations CRUD (Create, Read, Update, Delete)

Créer des templates en utilisant Thymeleaf

Implémenter la connexion/déconnexion en utilisant Spring Security

```java
@Entity  7 usages
@Data
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Enumerated(EnumType.STRING)
    @NotNull(message = "Product Type can not be null")
    private ProductType productType;

    @NotBlank(message = "Bran can not be null")
    private String brand;

    @NotBlank(message = "Model can not be null")
    private String model;

    @NotNull(message = "Price can not be null")
    @Positive
    private Double price;

    @NotNull(message = "Year can not be null")
    private Year year;
}
```

```java
public enum ProductType {  6 usages
    SMARTPHONE, TABLET, LAPTOP  no usages
}
```

application.yaml

```yaml
spring:
  application:
    name: spring-web-thymeleaf
  datasource:
    url: jdbc:mysql://localhost:3306/tech_store
    username: root
    password:
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate.ddl-auto: update
    properties:
      hibernate:
        dialect: org.hibernate.dialect.MySQLDialect
```

ProductRepository.java

```java
package org.uasz.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.uasz.model.Product;

public interface ProductRepository extends JpaRepository<Product, Long> {  2 usages
}
```

```java
@Controller  no usages
@RequiredArgsConstructor
public class ProductController {

    private final ProductRepository productRepository;

    @GetMapping("/")  no usages
    public String index(Model model){...}

    @GetMapping("/product/new")  no usages
    public String newProduct(Model model){...}

    @PostMapping("/save")  no usages
    public String saveProduct(@Valid @ModelAttribute("product") Product product, BindingResult bindingResult, Model model){...}

    @GetMapping("/product/edit/{id}")  no usages
    public String editProduct(@PathVariable Long id, Model model){...}

    @PostMapping("/update/{id}")  no usages
    public String updateProduct(@PathVariable Long id,
                                @Valid @ModelAttribute("product") Product product, BindingResult bindingResult, Model model){...}

    @GetMapping("/product/delete/{id}")  no usages
    public String deleteProduct(@PathVariable Long id){...}
}
```

```java
@GetMapping("/")  no usages
public String index(Model model){
    model.addAttribute( attributeName: "products", productRepository.findAll());
    return "index";
}
```

```html
<!DOCTYPE html>
<html lang="en" xmlns:th="https://www.thymeleaf.org/">
<head>
    <meta charset="UTF-8" name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Tech Store</title>
    <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css" />

    <script>
        function confirmDelete() {
          return confirm("Are you sure you want to delete this item?");
        }
    </script>
</head>
```

```
<div class="container">
    <h1>Products</h1>
    <a href="/product/new" class="btn btn-primary">Add new Product</a>
    <a href="/logout" class="btn btn-secondary float-right">Logout</a>
    <table class="table mt-3">
        <thead>
            <tr>
                <th>ID</th>
                <th>Type</th>
                <th>Brand</th>
                <th>Model</th>
                <th>Price</th>
                <th>Year</th>
                <th>Actions</th>
            </tr>
        </thead>
        <tbody>
            <tr th:each="product, status : ${products}">
                <td th:text="${status.count}"></td>
                <td th:text="${product.productType}"></td>
                <td th:text="${product.brand}"></td>
                <td th:text="${product.model}"></td>
                <td th:text="${product.price}"></td>
                <td th:text="${product.year}"></td>
                <td>
                    <a th:href="@{/product/edit/{id}(id=${product.id})}" class="btn btn-success">Edit</a>
                    <a th:href="@{/product/delete/{id}(id=${product.id})}" class="btn btn-danger" onclick="return confirmDelete();">Delete</a>
                </td>
            </tr>
        </tbody>
    </table>
</div>
```

```java
@GetMapping("/product/new")  no usages
public String newProduct(Model model){
    model.addAttribute( attributeName: "product", new Product());
    model.addAttribute( attributeName: "types", ProductType.values());
    return "newProduct";
}

@PostMapping("/save")  no usages
public String saveProduct(@Valid @ModelAttribute("product") Product product, BindingResult bindingResult, Model model){
    if(bindingResult.hasErrors()){
        model.addAttribute( attributeName: "types", ProductType.values());
        return "newProduct";
    }
    productRepository.save(product);
    return "redirect:/";
}
```

```
<form th:action="@{/save}" th:object="${product}" method="POST">
    <div class="form-group">
        <label for="productType">Type</label>
        <select id="productType" th:field="*{productType}" class="form-control">
            <option value="" disabled>Select Product Type</option>
            <option th:each="type : ${types}" th:value="${type}" th:text="${type}">Type</option>
        </select>
        <div th:if="${#fields.hasErrors('productType')}" th:errors="*{productType}" class="text-danger"></div>
    </div>
    <div class="form-group">
        <label for="brand">Brand</label>
        <input type="text" id="brand" th:field="*{brand}" class="form-control"/>
        <div th:if="${#fields.hasErrors('brand')}" th:errors="*{brand}" class="text-danger"></div>
    </div>
    <div class="form-group">
        <label for="model">Model</label>
        <input type="text" id="model" th:field="*{model}" class="form-control"/>
        <div th:if="${#fields.hasErrors('model')}" th:errors="*{model}" class="text-danger"></div>
    </div>
    <div class="form-group">
        <label for="year">Year</label>
        <input type="number" id="year" th:field="*{year}" class="form-control"/>
        <div th:if="${#fields.hasErrors('year')}" th:errors="*{year}" class="text-danger"></div>
    </div>
    <div class="form-group">
        <label for="price">Price</label>
        <input type="number" step="0.01" id="price" th:field="*{price}" class="form-control"/>
        <div th:if="${#fields.hasErrors('price')}" th:errors="*{price}" class="text-danger"></div>
    </div>
    <button type="submit" class="btn btn-primary">Save</button>
    <a th:href="@{/}" class="btn btn-secondary"> Back to Product List</a>
        <a href="/" class="btn btn-secondary">Back to Home</a>-->
</form>
```

```java
@GetMapping("/product/edit/{id}")  no usages
public String editProduct(@PathVariable Long id, Model model){
    Product product = productRepository.findById(id).orElseThrow(() -> new IllegalArgumentException("Invalid Product id: " +id));
    model.addAttribute( attributeName: "product", product);
    model.addAttribute( attributeName: "types",ProductType.values());
    return "editProduct";
}

@PostMapping("/update/{id}")  no usages
public String updateProduct(@PathVariable Long id,
                            @Valid @ModelAttribute("product") Product product, BindingResult bindingResult, Model model){
    if(bindingResult.hasErrors()){
        model.addAttribute( attributeName: "types", ProductType.values());
        return "editProduct";
    }
    productRepository.save(product);
    return "redirect:/";
}
```

```
<td>
    <a th:href="@{/product/edit/{id}(id=${product.id})}" class="btn btn-success">Edit</a>
    <a th:href="@{/product/delete/{id}(id=${product.id})}" class="btn btn-danger" onclick="return confirmDelete();">Delete</a>
</td>
```

```
<label for="productType">Type</label>
<select id="productType" th:field="*{productType}" class="form-control">
    <option value="" disabled>Select Product Type</option>
    <option th:each="type : ${types}" th:value="${type}" th:text="${type}">Type</option>
</select>
<div th:if="${#fields.hasErrors('productType')}" th:errors="*{productType}" class="text-danger"></div>
</div>
```

```java
@GetMapping("/product/delete/{id}")  no usages
public String deleteProduct(@PathVariable Long id){
    productRepository.deleteById(id);
    return "redirect:/";
}
```

```html
<td>
    <a th:href="@{/product/edit/{id}(id=${product.id})}" class="btn btn-success">Edit</a>
    <a th:href="@{/product/delete/{id}(id=${product.id})}" class="btn btn-danger" onclick="return confirmDelete();">Delete</a>
</td>
```

```html
<script>
    function confirmDelete() {
      return confirm("Are you sure you want to delete this item?");
    }
</script>
```

Implémenter la connexion/déconnexion en utilisant Spring Security

```java
@Configuration  no usages
@EnableWebSecurity
public class SecurityConfig {

    @Bean  no usages
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception{
        return http
                .formLogin(Customizer.withDefaults())
                .build();
    }

    @Bean  no usages
    public UserDetailsService userDetailsService(){
        UserDetails user = User.withDefaultPasswordEncoder()
                .username("admin")
                .password("admin")
                .roles("ADMIN")
                .build();
        return new InMemoryUserDetailsManager(user);
    }
}
```

Exercice d’application

PROJET : GESTION DE PATRIMOINE TOURISITQUE

Activités logicielles

- CRUD complet des sites (monuments, parcs, musées) ; Géolocalisation (coordonnées GPS) ;
- Upload et gestion des médias (photos, vidéos) ;
- Recherche avancée (par ville, type, période d'ouverture) ;
- Création d'une visite (date, heure, guide assigné) ;
- Réservation de places (avec gestion du stock/périmètre) ;
- Confirmation et annulation (logique métier) ;
- Génération de QR Code pour les tickets de visite ;
- Dépôt d'avis sur un site ou un guide ;
- Notation sur 5 étoiles ;
- Calcul automatique des notes moyennes ;
- Signalement d'avis inappropriés (modération) .

Cas d'utilisation : conservateur

- Valider les comptes guides
- Ajouter, modifier ou supprimer les sites;
- Modifier les horaires d'ouverture;
- Planifier une visite;

Cas d'utilisation : guide

- Créer un compte guide (avec demande de validation);
- Gérer le profil professionnel;
- Consulter l’agenda personnel du jour;
- Consulter ses avis et répondre aux commentaires ;
- Signaler un avis abusif

Cas d'utilisation : Touriste

- Se créer un compte, se connecter ;
- Rechercher des sites par proximité (GPS) ;
- Consulter les détails d'un site ;
- Réserver une visite, annuler la réservation ;
- Noter et commenter sa visite

Entités principales (modèle de données)

- `User (id, nom, email, password_hash, role [TOURIST, GUIDE, ADMIN])`
- `GuideDetails (id_user, numero_licence, biographie, note_moyenne, est_verifie)`
- `Site (id, nom, description, adresse, latitude, longitude, ville, horaires, capacite_accueil)`
- `Media (id, url, type [IMAGE/VIDEO], site_id)`
- `Visite (id, site_id, guide_id, date_heure_debut, duree, places_max, places_disponibles)`
- `Reservation (id, touriste_id, visite_id, statut [CONFIRME, ANNULE], qr_code_hash)`
- `Avis (id, reservation_id, note, commentaire, date_publication)`
- `Signalement (id, avis_id, motif, statut [EN_ATTENTE, RESOLU])`

Fin